SLSC: Sparse Logic State Compression for Multi-Turn Mathematical Reasoning

Anonymous ACL submission

Abstract

Multi-turn mathematical reasoning with small language models (SLMs, ∼1B–10B parameters) suffers from substantial performance degradation as conversation history accumulates, with recent work documenting up to 39% accuracy drops over extended interactions. We address this challenge by proposing *Sparse Logic State Compression* (SLSC), a twin-agent framework that maintains reasoning state as atomic mathematical propositions rather than unstructured text. A reasoning agent generates natural-language chains of thought while a curator agent distills accumulated context into structured propositions, replacing raw history with a compressed, addressable state. We evaluate SLSC on Math-100-multi, a 100-problem benchmark derived from MATH-500 where each problem decomposes into 2–5 dependent sub-questions requiring numerical results from prior turns. Across four Qwen3.5 scales (0.8B–9B), we find that compression methods substantially outperform raw history accumulation at 4B and above: SLSC achieves up to 85% fully correct rate versus 62% for uncompressed baselines, while reducing per-turn input tokens by 62%. However, natural-language summary shows comparable or larger gains, suggesting benefits arise primarily from bounded context size rather than structured representation. A cautionary finding emerges from Lean 4 formalization experiments: augmenting propositions with formal type stubs consistently degrades accuracy, attributed to stateless autoformalization without verifier feedback. Our work establishes structured state compression as an effective, training-free strategy for multi-turn mathematical reasoning in SLMs, while revealing that the core mechanism is preventing attention dilution rather than providing typed mathematical structure.

1 Introduction

Multi-turn interaction is rapidly becoming the dominant deployment mode for language models, yet recent work documents substantial performance degradation: (Laban et al., 2025) report an average 39% accuracy drop across six generation tasks over 200,000 conversations with 15 frontier LLMs. We study a specific instance of this phenomenon: *multi-turn mathematical reasoning with explicit numerical dependencies*, where each sub-question depends on the numerical result of the preceding turn. This setting is particularly acute for small language models (SLMs, ∼1B–10B parameters), whose limited attention capacity makes them vulnerable to compounding errors across turns.

The central question is how to maintain mathematical intermediate state across turns. Current approaches store state as unstructured text: prompt compression methods like LLMLingua (Jiang et al., 2023b) operate at the token level, agnostic to mathematical structure; reasoning-summarization methods like Reflexion (Shinn et al., 2023) and InftyThink (Yan et al., 2025) condense reasoning into natural-language narratives where numerical constraints receive no privileged representation. Raw context accumulation defers the problem until attention can no longer locate relevant constraints (Liu et al., 2024). We argue these approaches assume reasoning state should be flat text—a poor match for mathematical reasoning, where state is intrinsically typed: invariants, active variables, binding equations, each with distinct roles in subsequent inference.

We propose *Sparse Logic State Compression* (SLSC), a twin-agent framework where a reasoning agent $\mathcal{A}$ generates natural-language chains of thought while a curator agent $\mathcal{B}$ distills accumulated context into atomic mathematical propositions. The reasoning agent's context is replaced by this structured state, giving each constraint a persistent, addressable location. Figure 1 contrasts the two representations.

We evaluate SLSC on Math-100-multi, a 100-problem multi-turn benchmark derived from

Baseline: Natural-Language Summary

Flat text; intermediate constraints embedded in prose.

Context passed to Agent $\mathcal{A}$ at Turn 3:

"To approach this problem, I first considered the divisors of $n + 7$ and $2n + 1$. By examining the linear combination $2(n + 7) - (2n + 1)$, I observed that this simplifies to 13, which means any common divisor d must divide 13. Since 13 is prime, the possible values are $d = 1$ or $d = 13$. I then proceeded to analyze the case $d = 13$, which requires $n + 7 \equiv 0 \pmod{13}$, leading to $n \equiv 6 \pmod{13}$. Combined with the bounds, I narrowed down to candidates $n = 6, 19, 32, \ldots$, verifying each against the secondary constraint . . ."

~185 tokens • no addressable structure • key fact mid-sentence

SLSC: Sparse Logic State

Atomic propositions; each independently addressable.

Context passed to Agent $\mathcal{A}$ at Turn 3:

1. d divides $n + 7$
2. d divides $2n + 1$
3. $2(n + 7) - (2n + 1) = 13$
4. d divides 13
5. $d \in \{1, 13\}$
6. If $d = 13$: $n \equiv 6 \pmod{13}$
7. Candidates: $n \in \{6, 19, 32, \ldots\}$

~55 tokens • each fact retrievable • persistent slot per constraint

Figure 1: Comparison of context representations at Turn 3 of a multi-turn mathematical reasoning task. **Left**: Natural-language summary embeds constraints in prose, requiring attention over unstructured text to locate key facts (highlighted). **Right**: SLSC maintains atomic propositions in numbered slots, giving each constraint a persistent and addressable location. SLSC reduces token count by ~70% while improving constraint retrievability.

MATH-500 (Hendrycks et al., 2021) (L3–L5), where each problem decomposes into 2–5 dependent sub-questions. We compare four strategies across four Qwen3.5 reasoner scales (Qwen Team, 2024) (0.8B–9B). At 4B and above, compression methods substantially outperform raw history accumulation, with SLSC achieving strong gains while reducing per-turn tokens by 62%. However, augmenting SLSC with Lean 4 formalization consistently degrades accuracy, suggesting stateless autoformalization without verifier feedback introduces noise.

We make three contributions:

1. **Dataset.** Math-100-multi, a 100-problem multi-turn benchmark with explicit numerical dependencies, derived from MATH-500 (L3–L5).
2. **Structured state compression improves accuracy at 4B+ scales.** SLSC achieves substantial accuracy gains over raw history accumulation at 4B and above, while reducing per-turn input tokens by 62%. Natural-language summary shows comparable or larger gains, suggesting benefits come from bounded context size rather than structured representation alone.
3. **Cautionary finding on formal augmentation.** Lean 4 type stubs consistently degrade accuracy across all scales, attributed to stateless autoformalization without verifier feedback.

2 Related work

Multi-turn reasoning degradation. Recent work documents systematic performance loss in multi-turn settings. Laban et al. (2025) report an average 39% accuracy drop across six generation tasks over 200,000 simulated conversations with 15 frontier models, characterizing failure as a "lost-in-conversation" phenomenon. Liu et al. (2024) show that language models do not robustly use information in long contexts, with retrieval accuracy varying sharply by position. These findings establish multi-turn degradation as broad-spectrum, but existing multi-turn benchmarks largely target underspecification and clarification rather than numerical chaining. Our dataset complements this literature by isolating numerical-dependency failures, where each sub-question is fully specified but its solution requires the prior turn's numerical result.

Prompt and token-level compression Prompt compression methods reduce input length while attempting to preserve task performance. LLMLingua (Jiang et al., 2023b) and its extensions identify and remove low-information tokens via small auxiliary models, operating at the lexical level without awareness of which tokens encode task-critical structure. By design, a numerical constant inside an inequality and a discourse adjective receive comparable treatment. SLSC differs along two axes: compression operates over se-

mantic units—atomic mathematical propositions—rather than tokens, and is performed by a dedicated curator agent prompted to identify mathematical content. We view the two approaches as orthogonal and do not include token-level compression in our experimental comparison, which focuses on state-maintenance strategies sharing the same multi-turn decomposed setup.

Reasoning-state compression and summarization A closer line of work compresses intermediate reasoning state rather than raw inputs. Reflexion (Shinn et al., 2023) condenses prior trajectories into natural-language reflections that condition subsequent attempts. InftyThink (Yan et al., 2025) extends this idea to long-horizon reasoning by iteratively summarizing chain-of-thought and re-prompt the model on the summary, achieving large context savings. Conceptually these methods sit closest to SLSC, and two differences are central. First, both produce flat natural-language summaries in which numerical constraints stated in passing receive no privileged representation; SLSC enforces an atomic-proposition schema in which each targets monolithic single-question reasoning with internally-induced summarization points and assumes a trained summarization checkpoint, whereas our setting is externally-decomposed multi-turn dialogue with explicit numerical dependencies and is fully training-free. A head-to-head empirical comparison would require either retraining InftyThink on our dependency-annotated dataset or rebuilding it as a prompted pipeline; we leave this to future work and restrict our experiments to training-free, setup-compatible baselines.

Implicit assumptions in compression literature The compression methods above share an implicit assumption: that reducing context length improves downstream task performance, either by mitigating attention dilution (Liu et al., 2024) or by enabling longer reasoning chains within fixed context budgets. This assumption is rarely tested in controlled settings where compressed and uncompressed strategies are compared on identical problems with statistical rigor. Our work directly evaluates this assumption in the multi-turn mathematical reasoning regime. We find that compression methods substantially outperform raw history accumulation in accuracy at 4B parameters and above (Section 5), while delivering substantial per-turn token savings. This positions compression as both an accuracy intervention and a serving-cost intervention, and suggests that prior claims of compression-driven accuracy gains are well-founded in this regime, though the benefit comes primarily from bounded context size rather than from structured representation per se.

Formal language augmentation for reasoning. A separate line of work explores formal-language augmentation for mathematical reasoning. Autoformalization (Wu et al., 2022) translates natural-language statements into formal expressions; Draft, Sketch, and Prove (Jiang et al., 2023a) interleaves natural-language drafting with formal verification; LeanDojo (Yang et al., 2023) provides infrastructure for retrieval-augmented theorem proving. These works target end-to-end proof construction or single-statement formalization, with verifier feedback in the loop. SLSC's Lean 4 augmentation (Section 3.3) instead explores a lighter use of formal language—type-annotating extracted propositions without a verifier— and our ablation (Section 5.2) finds consistent accuracy degradation. This negative result aligns with the broader emphasis on verifier-grounded formalization: absent a Lean kernel check, the type annotation introduces noise that the reasoner cannot reliably exploit. SLSC thus occupies a specific intersection in this landscape—training-free, multi-turn, mathematics-targeted, and operating on semantic units of intermediate state—with raw context accumulation and natural-language summary as the primary setup-compatible baselines.

3 Method

3.1 Overview

SLSC encodes the accumulated multi-turn reasoning state as a sequence of atomic mathematical propositions, maintained across turns by a curator agent. Given a mathematical problem P decomposed into sub-questions $q_1, \ldots, q_T$, we denote the compressed state at turn t as S_t. The framework operates in two phases per turn t: (1) **Reasoning**: Agent $\mathcal{A}$ generates response $r_t = \mathcal{A}(P, S_{t-1}, q_t)$, conditioned on the compressed state from the previous turn; (2) **Curation**: Agent $\mathcal{B}$ updates the state $S_t = \mathcal{B}(S_{t-1}, r_t)$. Figure **??** illustrates the overall architecture.

3.2 Agent A: The Reasoner

Agent $\mathcal{A}$ serves as the per-turn reasoning component. At each turn t, it receives a structured prompt

Algorithm 1 Agent $\mathcal{B}$ Curation: extract atomic propositions from response, optionally formalize each, then merge into compressed state.

Require: S_{t-1}, r_t *(current state, Agent $\mathcal{A}$ response)*
Ensure: S_t
$\mathcal{P}_t \leftarrow \textsc{Extract}(r_t)$ ▷ LLM-based proposition extraction
for each $p \in \mathcal{P}_t$ **do** *(optional)*
  $p.\texttt{lean} \leftarrow \textsc{Formalize}(p)$ ▷ Autoformalize to Lean 4
end for
$S_t \leftarrow \textsc{Merge}(S_{t-1}, \mathcal{P}_t)$ ▷ LLM-based deduplication and update
return S_t

comprising the original problem P, the current compressed state S_{t-1}, and the sub-question q_t, and produces a natural language response r_t:

$$r_t = \mathcal{A}(P,\ S_{t-1},\ q_t)$$

Agent $\mathcal{A}$ is the frequent-call component of the pipeline, invoked once per turn. Specific model instantiations are described in Section 4.

3.3 Agent B: The Curator

Agent $\mathcal{B}$ maintains a persistent compressed state by processing $\mathcal{A}$'s response through three sequential sub-steps: extraction, optional formalization, and merging. Algorithm 1 summarizes the pipeline.

3.3.1 Extraction

Given $\mathcal{A}$'s response r_t, the extraction step produces a set of atomic propositions $\mathcal{P}_t = \{p_1, \ldots, p_k\}$, where each p_i is a self-contained mathematical claim. We use an LLM-based extractor prompted to identify equations, inequalities, numeric results, and named mathematical properties, while ignoring procedural descriptions and section headers. This yields propositions in slot form without requiring the full reasoning trace.

The following box shows a representative extraction example on a divisibility problem:

Extraction Example

Input (excerpt from r_t): *"Since $d \mid n{+}7$ and $d \mid 2n{+}1$, we can compute $d \mid 2(n{+}7){-}(2n{+}1) = 13$. Therefore d must divide 13 ..."*

Extracted propositions $\mathcal{P}_t$:
1. d divides $n + 7$
2. d divides $2n + 1$
3. $2(n + 7) - (2n + 1) = 13$
4. d divides 13

3.3.2 Merging

The merging step integrates the newly extracted propositions $\mathcal{P}_t$ into the existing state S_{t-1} to produce S_t. We apply three LLM-prompted merge rules:

- **Deduplication**: semantically equivalent propositions are consolidated into a single entry.
- **Supersession**: a new proposition that contradicts or subsumes an existing entry replaces it.
- **Preservation**: propositions with no conflict are retained verbatim.

These rules prevent unbounded state growth while ensuring no established fact is silently overwritten. The merger is implemented as a single prompted call to $\mathcal{B}$; full prompt details appear in Appendix **??**.

3.3.3 Formalization

As an optional augmentation, each extracted proposition p may be autoformalized into a Lean 4 statement appended alongside its natural language form. We explore Lean 4 augmentation to assess whether formal-language grounding affects downstream reasoning. Concretely, $\mathcal{B}$ is prompted to produce a `theorem` stub for each proposition, with the proof obligation left as `sorry`. We treat this component as optional: our ablation study (Section **??**) shows that formalization consistently degrades accuracy, and all primary results are reported without it unless stated otherwise.

3.4 State Representation

The state S_t is rendered as a numbered list of natural language propositions, optionally paired with Lean 4 stubs, and passed directly to $\mathcal{A}$ as part of its prompt context. Each proposition is atomic and ordered by extraction sequence; inter-proposition dependencies are not encoded explicitly. The state has no hard size cap; deduplication during merging

Algorithm 2 SLSC Multi-Turn Inference: iterate over sub-questions, alternating Agent $\mathcal{A}$ reasoning and Agent $\mathcal{B}$ state curation, then extract the final answer.

Require: Problem P, sub-questions $\{q_1, \ldots, q_T\}$, agents $\mathcal{A}$, $\mathcal{B}$
Ensure: Final answer

$S_0 \leftarrow \emptyset$ ▷ empty initial state
for $t = 1$ **to** T **do**
  $r_t \leftarrow \mathcal{A}(P,\ S_{t-1},\ q_t)$ ▷ Agent $\mathcal{A}$ reasons
  $S_t \leftarrow \mathcal{B}(S_{t-1},\ r_t)$ ▷ Agent $\mathcal{B}$ updates state
end for
return EXTRACTANSWER(r_T)

controls growth in practice. The following example shows a typical S_t after three curation steps:

[Known facts from prior steps]

1. d divides $n + 7$
```
theorem prop1 : d | (n + 7) := by sorry
```
2. d divides $2n + 1$
```
theorem prop2 : d | (2*n + 1) := by sorry
```
3. $2(n + 7) - (2n + 1) = 13$
```
theorem prop3 : 2*(n+7) - (2*n+1) = 13 := by
sorry
```
4. d divides 13
```
theorem prop4 : d | 13 := by sorry
```

3.5 Inference Workflow

Algorithm 2 summarizes the end-to-end multi-turn inference procedure.

4 Experimental Setup

4.1 Dataset

We evaluate SLSC on Math-100-multi, a multi-turn extension of 100 problems randomly sampled from MATH-500 (Hendrycks et al., 2021), restricted to Levels 3–5. Original single-turn problems are decomposed into sequences of dependent sub-questions through an automated pipeline (described in Appendix **??**); each sub-question requires the numerical result of the preceding turn, producing chains of length 2 to 5. The dataset contains 13 two-turn problems, 58 three-turn problems, 26 four-turn problems, and 3 five-turn problems (average: 3.19 turns per problem). Full dataset statistics and decomposition methodology are provided in Appendix **??**.

4.2 Models

We instantiate Agent $\mathcal{A}$ with four sizes from the Qwen3.5 series (0.8B, 2B, 4B, 9B) to study how reasoner capacity interacts with structured state compression. Agent $\mathcal{B}$ is instantiated with Gemma3-27B throughout. The asymmetric instantiation reflects the differing computational profiles of the two roles: $\mathcal{A}$ is invoked once per turn on the hot path, while $\mathcal{B}$ runs once per turn with a higher per-call latency budget.

All models are served via vLLM. For both agents, we use temperature $= 0$, top-$p = 1.0$, and maximum generation length of 1024 tokens. We set `enable_thinking: True` for all Qwen3.5 calls to keep the configuration consistent across sizes. Detailed sampling parameters and prompt templates appear in Appendix **??**.

4.3 Baselines and Conditions

We compare four conditions sharing identical decoding configuration and evaluation pipeline:

- **No Compression (Full History)**: the standard multi-turn baseline. Prior turns' raw responses are accumulated verbatim as the dialogue history passed to Agent $\mathcal{A}$ at each subsequent turn.
- **Summary**: a natural-language summary baseline in the style of Reflexion (Shinn et al., 2023). Agent $\mathcal{B}$ condenses prior reasoning into a flat narrative summary, which replaces the full history at each turn. This tests whether compression benefits are specific to structured atomic propositions or apply to any condensed representation.
- **SLSC (No Formal)**: SLSC without Lean 4 augmentation. Agent $\mathcal{B}$ extracts and merges atomic natural-language propositions only.
- **SLSC (with Lean)**: full SLSC including optional autoformalization of each extracted proposition.

We discuss the relationship to iterative summarization frameworks such as InftyThink (Yan et al., 2025) in Section **??**; direct empirical comparison is precluded by the differing task setups (monolithic single-question reasoning vs. externally-decomposed multi-turn dialogue) and InftyThink's training requirement.

4.4 Evaluation Metrics

We report four metrics:

- **Fully Correct Rate (FCR)** (primary): fraction of problems for which all sub-question answers are correct.
- **Correct Turn Rate (CTR)** (secondary): fraction of sub-question turns (across all problems) answered correctly.
- **Per-turn Input Tokens**: average input token count seen by Agent $\mathcal{A}$ per turn.
- **Total Inference Tokens**: end-to-end token consumption per problem, including both $\mathcal{A}$ and $\mathcal{B}$ calls.

Answer correctness is determined by exact symbolic match after `sympy`-based normalization.

4.5 Statistical Analysis

We assess statistical significance using McNemar's test for paired binary outcomes (correct/incorrect per problem). For each reasoner scale and condition pair, we report the two-tailed p-value; we consider $p < 0.05$ as statistically significant. Given the small sample size ($n = 100$), we interpret $p < 0.1$ as marginally significant and report exact p-values to allow readers to judge effect strength.

4.6 Implementation Details

Code, prompts, and dataset construction scripts will be released upon publication. Detailed implementation specifications are provided in Appendix **??**.

5 Results and Analysis

5.1 Main Results: Compression Improves Accuracy at 4B and 9B

Table 1 reports our main results across four reasoner scales and four compression strategies. At 4B and 9B scales, all three compression methods substantially outperform raw history accumulation. At 4B, compression methods achieve 69–77% vs. 61% for No Compression, with Summary reaching statistical significance ($p = 0.004$) and SLSC showing marginal significance ($p = 0.099$ without Lean, $p = 0.201$ with Lean). At 9B, the pattern strengthens: compression methods achieve 80–85% vs. 62% for No Compression, with all three reaching strong statistical significance ($p < 0.001$ for SLSC w/o Formal and Summary, $p = 0.001$ for SLSC with Lean). At 2B, all compression strategies underperform No Compression (45–50% vs. 51%), though differences are not statistically significant. At 0.8B, SLSC methods show directional gains (20–21% vs. 12%) while Summary underperforms (9%), but absolute accuracy remains low across all conditions.

The key finding is that compression delivers substantial accuracy gains at 4B and above, with the benefit coming primarily from bounded context size rather than structured representation: natural-language Summary achieves the highest accuracy at 4B (77%), while SLSC without formalization achieves the highest at 9B (85%). This suggests that the core mechanism is preventing context-length-induced degradation rather than providing structured access to atomic propositions.

5.2 Lean Ablation: Consistent Degradation from Formal Augmentation

Table 1 shows that adding Lean 4 formalization to SLSC consistently degrades or leaves unchanged downstream accuracy across all four scales. The mean effect is -2.75 percentage points; at 2B the degradation is severe (-5pp, from 50% to 45%). At 0.8B, Lean augmentation produces a small positive effect ($+1$pp, from 20% to 21%); at 4B and 9B, the effect is negative (-2pp and -5pp respectively). Across all scales, Lean augmentation never improves accuracy beyond the 2B threshold, and consistently degrades at the scales where SLSC shows its strongest gains (4B, 9B).

We manually annotated 50 randomly sampled Lean outputs produced by Agent $\mathcal{B}$ during the Qwen3.5-4B run (Table 2). Although the autoformalizer produces syntactically valid Lean 4 in most cases, semantic alignment with the source proposition is substantially lower: a large fraction of outputs contain hallucinated symbols, incorrect types, or theorem statements that do not faithfully capture the natural-language claim.

We identify four contributing factors to the Lean ablation result:

1. **Out-of-distribution autoformalization**: statement-level autoformalizers are trained on problem-theorem pairs, not on context-stripped atomic intermediate propositions.
2. **No verifier feedback**: SLSC's autoformalization is stateless; there is no Lean kernel check that could reject malformed output. Without verifier-in-the-loop feedback, the formalized output introduces noise rather than grounding.
3. **Reasoner under-utilization**: qualitative inspection of Agent $\mathcal{A}$ responses shows the reasoner rarely references the Lean stubs explicitly when constructing its answer.
4. **Granularity mismatch**: atomic propositions

Table 1: Main results across reasoner scales on Math-100-multi ($n = 100$ problems). FCR = Fully Correct Rate (%). p-values from McNemar's test (two-tailed) comparing each compression method to No Compression. † marks $p < 0.1$ (marginally significant); * marks $p < 0.05$; ** marks $p < 0.01$; *** marks $p < 0.001$.

Reasoner	**No Comp.**	**Summary**	**SLSC w/o Formal**	**SLSC (ours)**	**p-values vs. No Comp.**		
					Summary	SLSC w/o F.	SLSC
Qwen3.5-0.8B	12.0	9.1	20.0	21.0	0.804	0.134	$0.078^{\dagger}$
Qwen3.5-2B	51.0	46.0	50.0	45.0	0.458	1.000	0.362
Qwen3.5-4B	61.0	77.0	71.0	69.0	0.004^{**}	$0.099^{\dagger}$	0.201
Qwen3.5-9B	62.0	82.0	85.0	80.0	0.001^{***}	0.000^{***}	0.001^{***}

Table 2: Manual quality analysis of 50 randomly sampled Lean 4 outputs from SLSC + Lean (Qwen3.5-4B).

Quality dimension	**Count**	**%**
Syntactically valid Lean 4	12	24.0
Semantically aligned with NL	50	100.0
Hallucinated symbols/types	28	56.0
Successfully type-checks	12	24.0

stripped of problem context may lack the semantic richness needed for meaningful formalization.

This negative result has broader implications for formal-language augmentation in LLM reasoning pipelines: stateless autoformalization without verifier feedback appears insufficient to improve downstream reasoning, even when the formal language is syntactically correct. The finding aligns with prior work emphasizing verifier-grounded formalization (Jiang et al., 2023a; Yang et al., 2023), and suggests that the benefit of formal language comes from the verifier's rejection signal rather than the formal syntax itself.

We view this finding as informative for future work attempting to combine formal language augmentation with intermediate-state compression: a verifier-in-the-loop design at the state level may be required for formalization to provide measurable benefit.

5.3 Token Efficiency: Accuracy Gains with Reduced Per-Turn Cost

Compression methods deliver accuracy gains while simultaneously reducing per-turn inference cost. Table 3 reports token usage at the 4B scale. Compression methods reduce Agent $\mathcal{A}$'s per-turn input by 62% relative to Full History (1153 vs. 3058 tokens). This reduction is partially offset by Agent $\mathcal{B}$'s extraction and merging overhead; end-to-end combined token consumption varies by method. Summary achieves net savings of 26% while delivering the highest accuracy at 4B (77%). SLSC without formalization achieves 16% net savings with 71% accuracy. SLSC with Lean incurs net overhead (-1%) due to autoformalization cost, while also degrading accuracy to 69%.

Table 3: Token usage per problem at the Qwen3.5-4B scale. $\mathcal{A}$ Input averages across all turns. $\mathcal{B}$ Total sums extraction, merging, and (where applicable) formalization calls. Combined includes both agents' input and output tokens.

Condition	**$\mathcal{A}$ In**	**$\mathcal{A}$ Out**	**$\mathcal{B}$ Tot.**	**Comb.**
Full History	3058	2110	0	5168
Summary	1153	1987	698	3838
SLSC w/o Formal	1153	2034	1142	4329
SLSC (ours)	1153	2041	2038	5232

Note: Net savings vs. Full History: Summary 26%, SLSC w/o Formal 16%, SLSC (ours) -1%.

Figure 2 visualizes per-turn state size growth. Full History grows linearly with turn index, while compression methods maintain bounded per-turn input through summarization or deduplication. The key finding is that compression delivers both accuracy gains and per-turn token savings at 4B and above: Summary achieves 77% accuracy with 62% per-turn input reduction and 26% net token savings; SLSC without formalization achieves 71% accuracy with 62% per-turn input reduction and 16% net savings. Both substantially outperform Full History's 61% accuracy at higher total cost.

Cost-accuracy tradeoff. Compression methods shift cost from frequent Agent $\mathcal{A}$ calls (on growing contexts) to periodic Agent $\mathcal{B}$ curation calls. Summary achieves the best cost-accuracy tradeoff at 4B (highest accuracy, largest net savings). SLSC without formalization provides moderate net savings with strong accuracy gains. SLSC with Lean augmentation incurs net overhead while degrading accuracy, making it strictly dominated. The choice between Summary and SLSC depends on whether natural-language narrative or structured atomic propositions better match downstream rea-

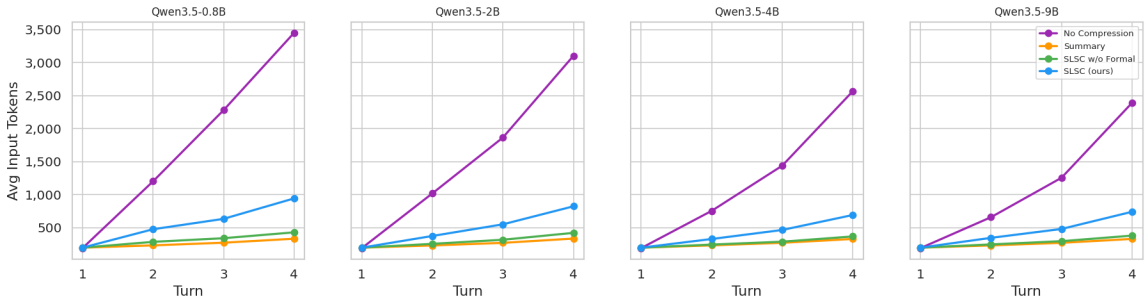

Figure 2: Cumulative input tokens seen by Agent $\mathcal{A}$ as a function of turn index across all Qwen3.5 scales (0.8B, 2B, 4B, 9B). Full History grows linearly across turns, while compression methods maintain near-constant per-turn state size.

Table 4: Failure mode distribution of Full History on Qwen3.5-4B (31 problems where compression methods achieve higher per-turn accuracy). Categories are not mutually exclusive; rows may sum to >100%.

Failure mode	**Count**	**%**
Response-style cascade	7	22.6
Answer truncation at `max_tokens`	23	74.2
Numerical condition forgetting	1	3.2
Logic error unrelated to context	0	0.0
Other	0	0.0

soning requirements; our results suggest both are effective, with Summary showing a slight edge at 4B and SLSC showing a slight edge at 9B.

5.4 Failure Mode Analysis: Understanding Compression Benefits and Limitations

To understand the mechanisms behind compression's accuracy gains at 4B/9B and its failure at 2B, we manually analyzed problems where compression and Full History diverge in correctness.

5.4.1 Why does compression help at 4B?

We annotated 31 problems on which Full History fails but at least one compression method achieves higher per-turn accuracy, at the 4B scale. Table 4 reports the failure mode distribution.

The dominant failure mode is **answer truncation at `max_tokens`** (74%): Full History accumulates verbatim prior turns, which inflates the input context and forces the model to use more tokens to re-read prior work before generating; combined with the 1024-token per-turn output cap, the model runs out of space mid-derivation. A secondary pattern is **response-style cascade** (23%): Agent $\mathcal{A}$'s prior-turn responses, when accumulated in dialogue history, induce the model to produce increasingly verbose meta-reasoning trace in subsequent turns. Compression methods avoid both failures by maintaining only the essential mathematical facts, allowing the reasoner to produce direct answers within the token budget.

Case study. Figure 3 shows a representative example. By turn 4, the Full History condition has accumulated 1681 tokens of input context from prior turns; Agent $\mathcal{A}$ produces a verbose checklist-style response that exhausts the 1024-token generation budget before reaching the final numerical answer. In contrast, SLSC maintains only 229 tokens of compressed state, allowing the reasoner to produce a direct derivation and correct answer within the token limit.

5.4.2 Why does compression fail at 2B?

We similarly annotated 16 problems where compression fails but Full History succeeds, at the 2B scale (Table 5). The dominant pattern is qualitatively different from 4B: 2B reasoners can parse the compressed state structure (0% parse failures) but struggle to correctly interpret its semantic content. In 81% of failures, the model encodes a wrong intermediate result in the state as early as turn 1 (e.g., wrong direction vector, wrong angle relation), and all subsequent turns read that incorrect fact as ground truth, compounding the error. In 12% of cases, the reasoner ignores the compressed state entirely and re-derives answers from the original problem statement only, losing the benefit of accumulated intermediate facts. This suggests a capability threshold: compressed representations require not just syntactic parsing ability, but semantic reasoning capacity to validate and correctly propagate intermediate facts—a capability that 2B models do not reliably exhibit, even though they handle short raw conversation histories adequately.

This pattern supports our capability-threshold interpretation: compressed representations require

Full History (Qwen3.5-4B) – Failure

Problem: Find largest integer < 2010 with remainder 5 (mod 7), 10 (mod 11), 10 (mod 13).

Turn 1 (137 tokens input)
"*Therefore, any number satisfying both conditions can be written as $x = 143k + 10$. If $k = 0$, $x = 10$. The smallest positive integer is **10**.*" ✓ Correct

Turn 2 (712 tokens input)
"$143k + 10 \equiv 5 \pmod 7$. *Since* $143 \equiv 3 \pmod 7$ *and* $10 \equiv 3 \pmod 7$, *we have* $3k + 3 \equiv 5 \pmod 7$, *so* $3k \equiv 2 \pmod 7$. *Testing: $k = 3$ gives $3 \times 3 = 9 \equiv 2$. Thus $k \equiv 3 \pmod 7$, smallest $k = 3$: $x = 143(3) + 10 = $ **439**.*" ✓ Correct

Turn 3 (1347 tokens input)
"*$lcm(7, 11, 13) = 7 \times 11 \times 13 =$ **1001**. The solutions form an arithmetic progression with common difference 1001. Adding 1001 to 439 gives the next solution (1440), etc.*" ✓ Correct

Turn 4 (1681 tokens input)
"***Constraint Checklist & Confidence Score:***
1. Answer q4? Yes.
2. Concise? Yes.
3. Clear? Yes.
4. Show reasoning? Yes.
5. Confidence Score: 100%."

× **GENERATION BUDGET EXHAUSTED**
Output contains only self-assessment checklist
No numerical answer produced

Final answer: None Token growth: 137→1681

SLSC (Qwen3.5-4B) – Success

Problem: Find largest integer < 2010 with remainder 5 (mod 7), 10 (mod 11), 10 (mod 13).

Turn 1 (142 tokens input)
"*Therefore, any number satisfying both conditions can be written as $x = 143k + 10$. To find the smallest positive integer, if $k = 0$, $x = 10$. **Answer: 10**.*" ✓ Correct
State S_1: `1. x = 143k + 10; 2. Smallest is 10` (54 tokens)

Turn 2 (188 tokens input)
"*The smallest positive integer is **439**.*" ✓ Correct
State S_2: `1. x = 143k + 10; 2. Answer is 439` (66 tokens)

Turn 3 (222 tokens input)
"*Therefore, the common difference is **1001**. Any integer satisfying the given remainder conditions can be found by adding multiples of 1001 to the base solution 439.*" ✓ Correct
State S_3: `1. x = 143k + 10; 2. Answer is 439; 3. Common diff is 1001` (96 tokens)

Turn 4 (229 tokens input)
"*The largest integer less than 2010 in this sequence is obtained when $k = 1$: $439 + 1001 = 1440$. **Answer: 1440**.*" ✓ Correct

✓ **ANSWER PRODUCED**
Direct computation with compact state
All turns correct

Final answer: 1440 Token growth: 142→229 (86% reduction)

Figure 3: Representative failure case at Qwen3.5-4B (problem: number_theory/1002). **Left:** Full History accumulates verbose reasoning across turns; by Turn 4, input context reaches 1681 tokens and the generation budget is exhausted, producing only a self-assessment checklist with no numerical answer (response-style cascade). **Right:** SLSC maintains compressed state (54–121 tokens); Turn 4 input is only 229 tokens, allowing the model to perform direct computation and produce the correct answer. Token reduction: 86%.

Table 5: Failure mode distribution of compression methods on Qwen3.5-2B (16 problems where Full History succeeds, compression fails).

Failure mode	Count	%
Fails to parse compressed representation	0	0.0
Ignores state, re-derives from P	2	12.5
Misinterprets state semantics	13	81.3
Other	1	6.3

not just syntactic parsing ability, but semantic reasoning capacity to validate and correctly use intermediate facts. 2B models can read the compressed state structure but cannot reliably verify its correctness or prevent error propagation across turns. At 4B and above, this threshold is crossed, and compression becomes beneficial.

5.5 Per-turn Accuracy Breakdown

Figure 4 shows per-turn Correct Turn Rate across all four Qwen3.5 scales. The capability threshold is clearly visible: at 0.8B and 2B, compression methods (especially SLSC with Lean) show sharp accuracy drops across turns, while Full History maintains relatively stable performance. At 4B and 9B, the pattern reverses: compression methods maintain stable accuracy across turns while Full History drops sharply at turns 3 and 4. This crossover supports our interpretation that compressed representations require a minimum reasoning capacity to parse and utilize effectively, and that above this threshold, compression prevents the cumulative cascade effects that degrade Full History performance.

6 Conclusion

We introduced Sparse Logic State Compression (SLSC), a twin-agent framework that addresses multi-turn mathematical reasoning degradation in small language models by maintaining accumulated context as atomic mathematical propositions rather than unstructured text. Through controlled experiments on Math-100-multi, a 100-problem benchmark with explicit numerical dependencies, we demonstrated that structured state compression substantially improves accuracy at 4B parameters and above, achieving strong gains over raw history accumulation while reducing per-turn input tokens by 62%.

Our results reveal a capability threshold: compression methods benefit models at 4B+ scales but harm 2B models, suggesting that parsing compressed representations requires a minimum rea-

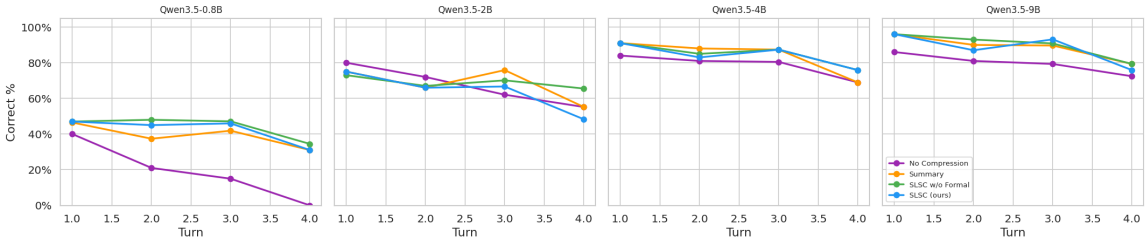

Figure 4: Per-turn Correct Turn Rate across all Qwen3.5 scales (0.8B, 2B, 4B, 9B). Sample sizes by turn: $n_1 = n_2 = 100$, $n_3 = 87$, $n_4 = 30$, $n_5 = 3$. We omit turn 5 from analysis due to small sample. The capability threshold is evident: compression methods harm accuracy at 0.8B/2B but substantially improve it at 4B/9B, with the gap widening at later turns.

soning capacity. Interestingly, natural-language summary achieves comparable or larger gains than SLSC's structured propositions, indicating that the primary benefit comes from bounded context size rather than structured representation alone. This finding challenges the assumption that mathematical reasoning inherently requires typed state, and suggests that attention dilution in long contexts is the dominant failure mode in this regime.

A cautionary finding emerged from our Lean 4 augmentation experiments: appending formal type stubs to extracted propositions consistently degraded accuracy across all scales. We attribute this to stateless autoformalization without verifier feedback, which introduces syntactic noise that small models cannot reliably exploit. This negative result underscores the importance of verifier-grounded formalization and suggests that lightweight formal augmentation, while conceptually appealing, may not transfer benefits without a proof-checking loop.

Limitations and future work. Our evaluation is restricted to a single dataset of 100 problems and a single model family (Qwen3.5). The generalizability of the capability threshold and the relative performance of compression strategies should be validated on larger benchmarks and diverse model architectures. The dataset construction pipeline relies on automated decomposition, which may introduce artifacts; human-authored multi-turn problems would provide a stronger test. Additionally, our curator agent operates without verifier feedback; integrating a Lean kernel or symbolic algebra system to validate extracted propositions could address the formalization degradation we observed. Finally, the interaction between compression and other multi-turn interventions—such as retrieval-augmented memory or learned summarization—remains unexplored.

Despite these limitations, our work establishes that structured state compression is a viable and effective strategy for multi-turn mathematical reasoning in small language models, and provides a training-free framework that can be deployed immediately. The Math-100-multi dataset and our experimental pipeline offer a controlled testbed for future research on multi-turn reasoning degradation, and our findings on the capability threshold and formalization pitfalls provide actionable guidance for practitioners deploying small models in multi-turn settings.

Limitations

This document does not cover the content requirements for ACL or any other specific venue. Check the author instructions for information on maximum page lengths, the required "Limitations" section, and so on.

References

Dan Hendrycks, Collin Burns, Saurav Kadavath, Akul Arora, Steven Basart, Eric Tang, Dawn Song, and Jacob Steinhardt. 2021. Measuring mathematical problem solving with the MATH dataset. *arXiv preprint arXiv:2103.03874*.

Albert Q. Jiang, Sean Welleck, Jin Peng Zhou, Wenda Li, Jiacheng Liu, Mateja Jamnik, Timothée Lacroix, Yuhuai Wu, and Guillaume Lample. 2023a. Draft, sketch, and prove: Guiding formal theorem provers with informal proofs. *arXiv preprint arXiv:2210.12283*.

Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023b. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13358–13376. ArXiv:2310.05736.

Philippe Laban, Hiroaki Hayashi, Yingbo Zhou, and Jennifer Neville. 2025. LLMs get lost in multi-turn conversation. *arXiv preprint arXiv:2505.06120*.

Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12:157–173. ArXiv:2307.03172.

Qwen Team. 2024. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*.

Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. *Advances in Neural Information Processing Systems*, 36. ArXiv:2303.11366.

Yuhuai Wu, Albert Q. Jiang, Wenda Li, Markus N. Rabe, Charles Staats, Mateja Jamnik, and Christian Szegedy. 2022. Autoformalization with large language models. *arXiv preprint arXiv:2205.12615*.

Yuchen Yan, Yongliang Shen, Yang Liu, Jin Jiang, Mengdi Zhang, Jian Shao, and Yueting Zhuang. 2025. InftyThink: Breaking the length limits of long-context reasoning in large language models. *arXiv preprint arXiv:2503.06692*.

Kaiyu Yang, Aidan M. Swope, Alex Gu, Rahul Chalamala, Peiyang Song, Shixing Yu, Saad Godil, Ryan Prenger, and Anima Anandkumar. 2023. LeanDojo: Theorem proving with retrieval-augmented language models. In *Advances in Neural Information Processing Systems*, volume 36. ArXiv:2306.15626.

A MATH-100-multi Dataset Construction Prompts

This appendix provides the complete prompts used to construct the MATH-100-multi dataset, including the System Prompt and User Template.

A.1 System Prompt

```
You are a math problem decomposer. Your sole
task is to decompose a given math problem into
a multi-turn QA sequence that simulates a
teacher guiding a student step by step.

### Decomposition Rules

1. **Determine n automatically** based on
   solution complexity:
   - 1-2 key computation steps -> n = 2
   - 3-4 key computation steps -> n = 3~4
   - 5+ steps or multi-concept problems
     -> n = 4~6
   - Never inflate n with trivial or
     redundant sub-questions.

2. **Strict numerical carry-over dependency**:
   - Each q_i (i >= 2) must explicitly
     reference the numerical result produced
     by q_{i-1}.
   - Phrase q_i so the dependency is visible
     in the question text itself, e.g.:
     "Using the value of X you found in the
     previous step, ..."
   - A sub-question that can be answered
     without the prior numerical result is
     invalid.

3. **Coverage and chain completeness**:
   - The sub-questions q1 -> q2 -> ... -> qn
     must form a COMPLETE and CONTINUOUS
     reasoning chain that leads to the final
     answer.
   - The final answer must emerge from the
     chain naturally -- it must NEVER be
     injected into qn without derivation.
   - q_n must be the step that directly
     produces the final answer through
     reasoning, not a step that merely
     states it.

4. **Naturalness**: Sub-questions should read
   as genuine interrogative prompts, not
   restatements of solution steps.

5. **Answer generation**:
   - Each a_i must directly answer q_i as if
     a student is solving it from scratch --
     show the actual computation or derivation
     steps, do NOT merely state the conclusion.
   - Use the provided Solution as a reference
     to ensure correctness, but write each
     answer in the student's own working style:
       correct: "d/60 + d/40 = (2d+3d)/120
                = 5d/120 = 5, so d = 120 km."
       wrong:   "The distance is 120 km."
                (conclusion only)
   - a_i must include the specific numerical
     result that q_{i+1} will depend on,
     stated clearly at the end, e.g.:
       "Therefore, d = 120 km."
   - a_n must match the original problem's
     final Answer exactly.
   - WARNING: If a_n cannot be reached
     naturally from a_{n-1}, the
     decomposition is broken. Restructure
     from q1 -- do NOT patch qn to force
     the answer.

6. **Solution invisibility**:
   - The provided Solution is for YOUR
     reference only to understand the
     problem structure.
   - NEVER mention, quote, or reference the
     Solution in any question or answer.
   - Questions and answers must read as if
     the Solution does not exist.

### Output Format
Return **only** a valid JSON object. No
explanation, no markdown, no text outside
the JSON.

{
  "turns": <n>,
  "qa_pairs": [
    {"question": "q1: ...",
     "answer": "a1: ..."},
    {"question": "q2: ... (using the result
                  X from q1, ...)",
     "answer": "a2: ..."},
    ...
    {"question": "qn: ...",
     "answer": "an: ..."}
  ]
}
```

A.2 User Template

```
Please decompose the following math problem
into a multi-turn question sequence.

### Problem
{problem}

### Solution
{solution}

### Answer
{answer}
```

where {problem}, {solution}, and {answer} are replaced with the corresponding entries from the MATH-500 dataset.